processing

January 30, 2026

[270]:
```
import json

path = "location-history.json"
with open(path, "r", encoding="utf-8") as f:
    data = json.load(f)

type(data), len(data), data[0].keys()
```

[270]:
```
(list, 20021, dict_keys(['endTime', 'startTime', 'visit']))
```

[271]:
```
import pandas as pd
from datetime import timedelta
from dateutil import parser as dtparser
```

[272]:
```
def parse_geo(s: str):
    # "geo:33.896160,35.615430"
    s = s.replace("geo:", "")
    lat, lon = s.split(",")
    return float(lat), float(lon)

def duration_minutes(t0, t1):
    return (t1 - t0).total_seconds() / 60.0
```

0.0.1 extraction of data

[273]:
```
rows_visit, rows_act, rows_path = [], [], []

for i, ep in enumerate(data):
    if "startTime" not in ep or "endTime" not in ep:
        continue
    start = dtparser.isoparse(ep["startTime"])
    end   = dtparser.isoparse(ep["endTime"])
    dur = duration_minutes(start, end)
    if dur <= 0:
        continue

    # VISIT
    if "visit" in ep:
```

```
        v = ep["visit"]
        tc = v.get("topCandidate", {})
        loc = tc.get("placeLocation")
        if loc:
            lat, lon = parse_geo(loc)
            rows_visit.append({
                "episode_id": i,
                "start_time": start,
                "end_time": end,
                "duration_min": dur,
                "lat": lat,
                "lon": lon,
                "place_id": tc.get("placeID"),
                "semantic_type": tc.get("semanticType"),
                "visit_prob": float(v.get("probability", tc.get("probability",␣
↪1.0))),
            })

    # ACTIVITY
    if "activity" in ep:
        a = ep["activity"]

        # only process if we have the required geometry
        if "start" not in a or "end" not in a:
            continue

        slat, slon = parse_geo(a["start"])
        elat, elon = parse_geo(a["end"])

        tc = a.get("topCandidate", {})

        act_prob = a.get("probability", tc.get("probability"))
        act_prob = a.get("probability", tc.get("probability"))

        # normalize to float if possible
        try:
            act_prob = float(act_prob) if act_prob is not None else None
        except (TypeError, ValueError):
            act_prob = None

        # treat None or 0.0 as missing
        if act_prob is None or act_prob == 0.0:
            act_prob = 1.0


        rows_act.append({
            "episode_id": i,
```

```
            "start_time": start,
            "end_time": end,
            "duration_min": dur,
            "start_lat": slat,
            "start_lon": slon,
            "end_lat": elat,
            "end_lon": elon,
            "distance_m": float(a.get("distanceMeters", 0.0)),
            "mode_raw": tc.get("type"),
            "act_prob": float(act_prob),
        })


    # TIMELINE PATH POINTS
    if "timelinePath" in ep:
        for p in ep["timelinePath"]:
            plat, plon = parse_geo(p["point"])
            offset_min = float(p.get("durationMinutesOffsetFromStartTime", 0.0))
            t = start + timedelta(minutes=offset_min)
            rows_path.append({
                "episode_id": i,
                "time": t,
                "lat": plat,
                "lon": plon,
            })

visits = pd.DataFrame(rows_visit)
acts   = pd.DataFrame(rows_act)
pathpts= pd.DataFrame(rows_path)

visits.head(), acts.head(), pathpts.head()
```

[273]:
```
(   episode_id                       start_time  \
 0           0        2020-02-10 14:25:25+02:00
 1           2  2020-02-11 07:54:24.665000+02:00
 2           4  2020-02-11 15:40:17.483000+02:00
 3           6  2020-02-12 07:22:35.769000+02:00
 4           8  2020-02-12 15:43:38.316000+02:00

                          end_time  duration_min        lat        lon  \
 0  2020-02-11 06:50:26.122000+02:00    985.018700  33.896096  35.615478
 1  2020-02-11 14:56:37.618000+02:00    422.215883  33.918112  35.617579
 2  2020-02-12 06:53:03.972000+02:00    912.774817  33.896096  35.615478
 3  2020-02-12 15:02:42.828000+02:00    460.117650  33.918112  35.617579
 4  2020-02-13 17:29:52.864000+02:00   1546.242467  33.896096  35.615478

                      place_id semantic_type  visit_prob
```

```
0  ChIJqbmP24M-HxURAAAAAAAAAAA          Home        0.87
1  ChIJace_C_E-HxUR1jVhXa9WnpQ       Unknown        0.77
2  ChIJqbmP24M-HxURAAAAAAAAAAA          Home        0.89
3  ChIJace_C_E-HxUR1jVhXa9WnpQ       Unknown        0.76
4  ChIJqbmP24M-HxURAAAAAAAAAAA          Home        0.87  ,
   episode_id                       start_time  \
0           1  2020-02-11 06:50:26.122000+02:00
1           3  2020-02-11 14:56:37.618000+02:00
2           5  2020-02-12 06:53:03.972000+02:00
3           7  2020-02-12 15:02:42.828000+02:00
4           9  2020-02-13 17:29:52.864000+02:00

                         end_time  duration_min  start_lat  start_lon  \
0  2020-02-11 07:54:24.665000+02:00     63.975717  33.897085  35.615187
1  2020-02-11 15:40:17.483000+02:00     43.664417  33.918008  35.617143
2  2020-02-12 07:22:35.769000+02:00     29.529950  33.897099  35.615414
3  2020-02-12 15:43:38.316000+02:00     40.924800  33.917877  35.617327
4  2020-02-13 17:53:31.662000+02:00     23.646633  33.896209  35.615500

     end_lat    end_lon  distance_m             mode_raw  act_prob
0  33.917756  35.617426      4288.0  in passenger vehicle       1.0
1  33.896209  35.615501      2433.0          motorcycling       1.0
2  33.918377  35.616791      4948.0  in passenger vehicle       1.0
3  33.896209  35.615500      4790.0  in passenger vehicle       1.0
4  33.925053  35.587681      4108.0  in passenger vehicle       1.0  ,
   episode_id                      time        lat        lon
0       12983 2020-02-10 12:25:00+00:00  33.896189  35.615365
1       12984 2020-02-11 04:50:00+00:00  33.897085  35.615187
2       12984 2020-02-11 04:54:00+00:00  33.898311  35.612111
3       12984 2020-02-11 04:56:00+00:00  33.887462  35.621832
4       12984 2020-02-11 05:05:00+00:00  33.901157  35.609035)
```

[274]:
```
import pandas as pd

visits["start_time"] = pd.to_datetime(visits["start_time"], utc=True)
visits["end_time"]   = pd.to_datetime(visits["end_time"],   utc=True)

acts["start_time"] = pd.to_datetime(acts["start_time"], utc=True)
acts["end_time"]   = pd.to_datetime(acts["end_time"],   utc=True)

pathpts["time"] = pd.to_datetime(pathpts["time"], utc=True)
```

[275]:
```
visits["start_time"].dtype
acts["start_time"].dtype
pathpts["time"].dtype
```

[275]:
```
datetime64[us, UTC]
```

data cleaning (weighing)

[276]:
```
# basic sanity
visits = visits.dropna(subset=["lat","lon","duration_min"])
visits = visits[visits["duration_min"] > 0]

# optional: remove micro-stops
visits = visits[visits["duration_min"] >= 3]

# weight choice (probability-weighted dwell time)
visits["weight"] = visits["duration_min"] * visits["visit_prob"]
```

merging identical place_id's

[277]:
```
visits = visits.sort_values(["start_time"])
visits["prev_place"] = visits["place_id"].shift(1)
visits["prev_end"]   = visits["end_time"].shift(1)

# merge if same place and gap small (e.g., <= 10 min)
gap = (visits["start_time"] - visits["prev_end"]).dt.total_seconds()/60
merge_mask = (visits["place_id"].notna()) & (visits["place_id"] ==␣
 ↪visits["prev_place"]) & (gap <= 10)

# create group id
visits["group"] = (~merge_mask).cumsum()

visits = (visits.groupby("group", as_index=False)
          .agg({
              "episode_id":"first",
              "start_time":"min",
              "end_time":"max",
              "duration_min":"sum",
              "lat":"first","lon":"first",
              "place_id":"first",
              "semantic_type":"first",
              "visit_prob":"mean",
              "weight":"sum"
          }))
```

[278]:
```
acts = acts.
 ↪dropna(subset=["start_lat","start_lon","end_lat","end_lon","duration_min"])
acts = acts[acts["duration_min"] > 0]

# speed sanity check
acts["speed_mps"] = acts["distance_m"] / (acts["duration_min"]*60)
acts = acts[acts["speed_mps"] < 60]  # ~216 km/h, adjust if needed

# define weights (pick one)
```

```
acts["weight_dist"] = acts["distance_m"] * acts["act_prob"]    # corridor usage␣
 ↪by distance
acts["weight_time"] = acts["duration_min"] * acts["act_prob"] # corridor usage␣
 ↪by time
```

[279]:
```
def normalize_mode(m):
    if not isinstance(m, str):
        return "unknown"
    m = m.lower()
    if "walk" in m:
        return "walk"
    if "bicy" in m or "cycle" in m:
        return "bike"
    if "bus" in m or "train" in m or "subway" in m or "transit" in m:
        return "transit"
    if "vehicle" in m or "car" in m or "taxi" in m or "motor" in m:
        return "vehicle"
    return m

acts["mode"] = acts["mode_raw"].apply(normalize_mode)
```

[280]:
```
import osmnx as ox

city_name = "Beirut, Lebanon"
city_gdf = ox.geocode_to_gdf(city_name)
city_poly = city_gdf.geometry.iloc[0]
city_gdf
```

[280]:
```
                                            geometry  bbox_west  bbox_south  \
0  POLYGON ((35.46681 33.8932, 35.467 33.89315, 3…  35.466807   33.862331

   bbox_east  bbox_north   place_id  osm_type   osm_id        lat        lon  \
0  35.542581   33.916113  194355641  relation  5466662  33.889226  35.502558

     class     type  place_rank  importance addresstype    name  \
0  landuse  harbour          24    0.710529     harbour  Beirut

                                        display_name
0  Beirut, Basta Tahta, Bashura, Beirut Governora…
```

[281]:
```
import geopandas as gpd
from shapely.geometry import Point, LineString

# visits points
gvis = gpd.GeoDataFrame(
    visits,
    geometry=[Point(xy) for xy in zip(visits["lon"], visits["lat"])],
```

```
    crs="EPSG:4326"
)

# activities lines (straight line start->end)
gact = gpd.GeoDataFrame(
    acts,
    geometry=[LineString([(r.start_lon, r.start_lat), (r.end_lon, r.end_lat)])␣
 ↪for r in acts.itertuples()],
    crs="EPSG:4326"
)

# path points
gpath = gpd.GeoDataFrame(
    pathpts,
    geometry=[Point(xy) for xy in zip(pathpts["lon"], pathpts["lat"])],
    crs="EPSG:4326"
)

# pick a metric CRS automatically (UTM)
utm_crs = gvis.estimate_utm_crs()
gvis_m  = gvis.to_crs(utm_crs)
gact_m  = gact.to_crs(utm_crs)
gpath_m = gpath.to_crs(utm_crs)
city_m  = city_gdf.to_crs(utm_crs)
city_poly_m = city_m.geometry.iloc[0]
```

[282]:
```
gvis_m  = gvis_m[gvis_m.within(city_poly_m)]
gpath_m = gpath_m[gpath_m.within(city_poly_m)]
gact_m  = gact_m[gact_m.intersects(city_poly_m)]
```

[283]:
```
import numpy as np

cell = 100  # meters per pixel
minx, miny, maxx, maxy = city_poly_m.bounds

xs = np.arange(minx, maxx, cell)
ys = np.arange(miny, maxy, cell)
xx, yy = np.meshgrid(xs, ys)

grid_points = np.vstack([xx.ravel(), yy.ravel()]).T
```

[284]:
```
from sklearn.neighbors import KernelDensity

coords = np.vstack([gvis_m.geometry.x.values, gvis_m.geometry.y.values]).T
weights = gvis_m["weight"].values

bandwidth = 300  # meters: conceptual scale (try 200, 500, 1000 for sensitivity)
```

```
kde = KernelDensity(bandwidth=bandwidth, kernel="gaussian")
kde.fit(coords, sample_weight=weights)

log_dens = kde.score_samples(grid_points)
dens = np.exp(log_dens).reshape(xx.shape)
```

[285]:
```
from shapely.prepared import prep

prep_poly = prep(city_poly_m)
mask = np.array([prep_poly.contains(Point(p)) for p in grid_points]).reshape(xx.
 ↪shape)
dens_visits = np.where(mask, dens, np.nan)
```

[286]:
```
gpath_m = gpath_m.sort_values(["episode_id", "time"])
gpath_m["t_next"] = gpath_m.groupby("episode_id")["time"].shift(-1)
dt = (gpath_m["t_next"] - gpath_m["time"]).dt.total_seconds()/60.0
gpath_m["dt_min"] = dt.fillna(0)

# remove zeros and absurd gaps (optional)
gpath_m = gpath_m[(gpath_m["dt_min"] >= 0) & (gpath_m["dt_min"] <= 10)]

# weight: minutes spent near this point
gpath_m["weight"] = gpath_m["dt_min"]
```

[287]:
```
coords_p = np.vstack([gpath_m.geometry.x.values, gpath_m.geometry.y.values]).T
weights_p = gpath_m["weight"].values

bandwidth_move = 200
kde2 = KernelDensity(bandwidth=bandwidth_move, kernel="gaussian")
kde2.fit(coords_p, sample_weight=weights_p)

dens2 = np.exp(kde2.score_samples(grid_points)).reshape(xx.shape)
dens_move = np.where(mask, dens2, np.nan)
```

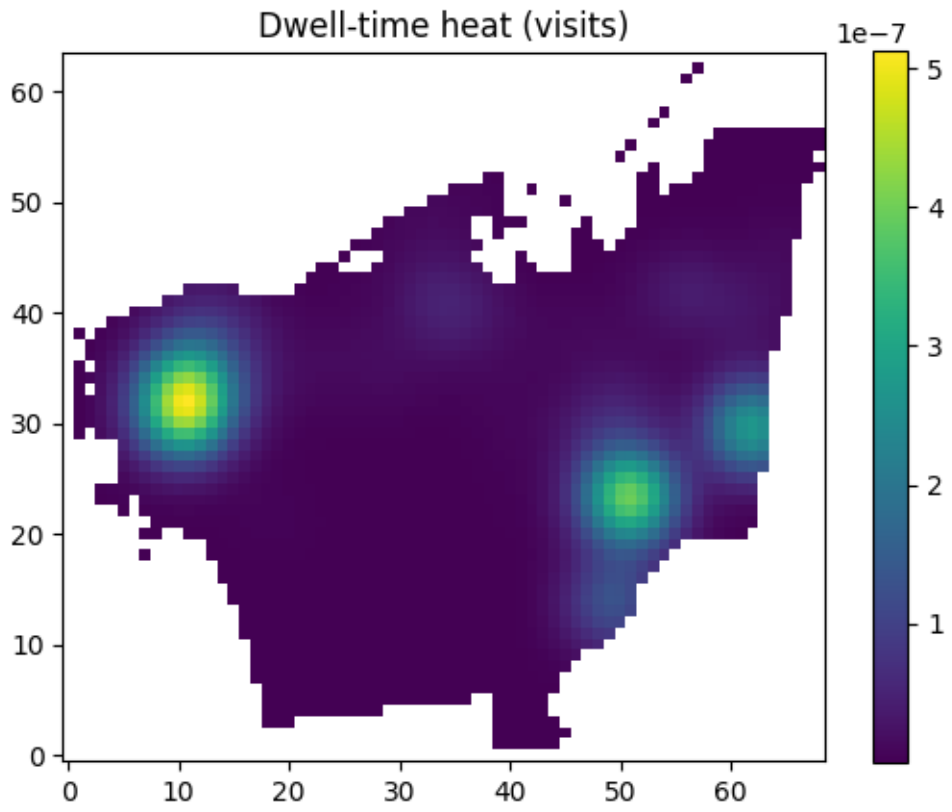
```
/home/elie/Desktop/files/map_proj/venv/lib/python3.11/site-
packages/sklearn/neighbors/_kde.py:278: RuntimeWarning: divide by zero
encountered in log
  log_density = self.tree_.kernel_density(
```

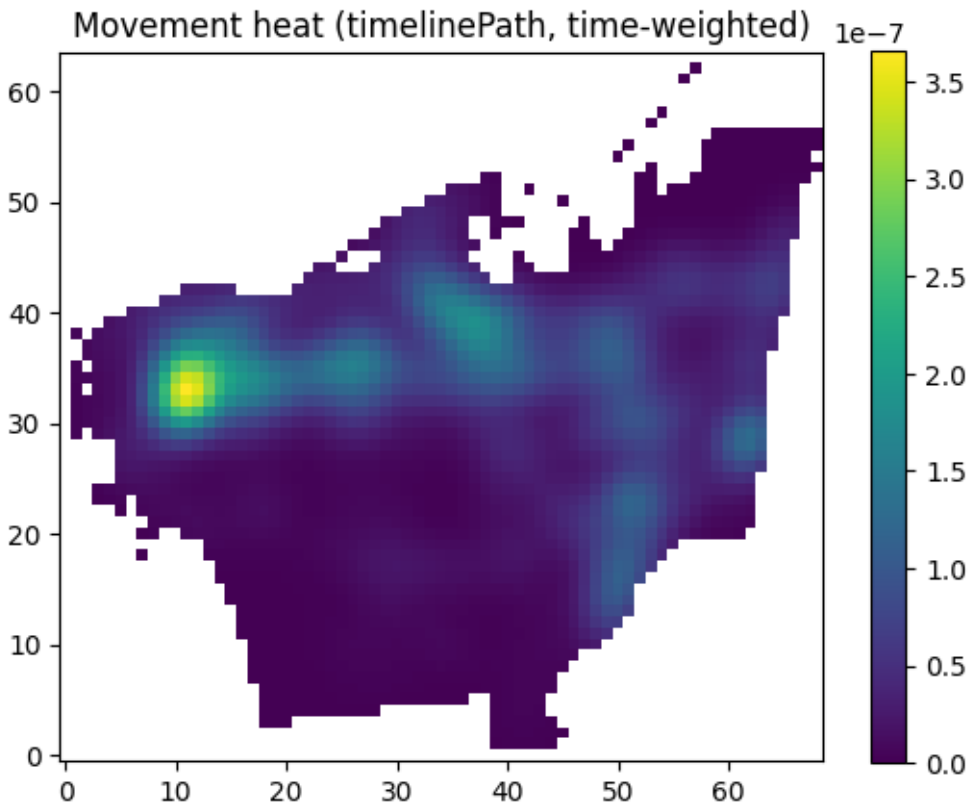
[288]:
```
episode_mode = acts.set_index("episode_id")["mode"]
gpath_m["mode"] = gpath_m["episode_id"].map(episode_mode).fillna("unknown")

def kde_surface(gdf_points, bw):
    coords = np.vstack([gdf_points.geometry.x.values, gdf_points.geometry.y.
 ↪values]).T
    w = gdf_points["weight"].values
    if len(coords) < 10:
```

```
        return np.full(xx.shape, np.nan)
    kde = KernelDensity(bandwidth=bw, kernel="gaussian")
    kde.fit(coords, sample_weight=w)
    dens = np.exp(kde.score_samples(grid_points)).reshape(xx.shape)
    return np.where(mask, dens, np.nan)

dens_walk = kde_surface(gpath_m[gpath_m["mode"]=="walk"], bw=200)
dens_vehicle = kde_surface(gpath_m[gpath_m["mode"]=="vehicle"], bw=300)
```

[289]:
```
import rasterio
from rasterio.transform import from_origin

transform = from_origin(minx, maxy, cell, cell)  # origin at top-left
height, width = dens_visits.shape

def write_geotiff(filename, arr):
    with rasterio.open(
        filename, "w",
        driver="GTiff",
        height=height, width=width,
        count=1,
        dtype="float32",
        crs=str(utm_crs),
        transform=transform,
        nodata=np.nan
    ) as dst:
        dst.write(arr.astype("float32"), 1)

write_geotiff("heat_visits_dwell.tif", dens_visits)
write_geotiff("heat_corridors_time.tif", dens_move)
write_geotiff("heat_corridors_walk.tif", dens_walk)
write_geotiff("heat_corridors_vehicle.tif", dens_vehicle)
```

[290]:
```
import matplotlib.pyplot as plt

plt.figure()
plt.imshow(dens_visits, origin="lower")
plt.title("Dwell-time heat (visits)")
plt.colorbar()
plt.show()

plt.figure()
plt.imshow(dens_move, origin="lower")
plt.title("Movement heat (timelinePath, time-weighted)")
plt.colorbar()
plt.show()
```


Dwell-time heat (visits)
1e−7
5
4
3
2
1
0
10
20
30
40
50
60



[291]:
```
import osmnx as ox
import geopandas as gpd
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Build a network. For "roads i frequent", drive is usually right.
# If you also want walking paths, use network_type="all" or build separate␣
 ↪graphs.
G = ox.graph_from_polygon(city_poly, network_type="drive", simplify=True)

# Project graph to metric CRS (usually UTM)
G = ox.project_graph(G)

# Edge GeoDataFrame (roads)
edges = ox.graph_to_gdfs(G, nodes=False, fill_edge_geometry=True)

edges.crs
```

[291]:
```
<Projected CRS: EPSG:32636>
Name: WGS 84 / UTM zone 36N
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: Between 30°E and 36°E, northern hemisphere between equator and 84°N,
onshore and offshore. Belarus. Cyprus. Egypt. Ethiopia. Finland. Israel. Jordan.
Kenya. Lebanon. Moldova. Norway. Russian Federation. Saudi Arabia. Sudan. Syria.
Türkiye (Turkey). Uganda. Ukraine.
- bounds: (30.0, 0.0, 36.0, 84.0)
Coordinate Operation:
- name: UTM zone 36N
- method: Transverse Mercator
Datum: World Geodetic System 1984 ensemble
- Ellipsoid: WGS 84
- Prime Meridian: Greenwich
```

[293]:
```
# Project to the graph CRS
gvis_p  = gvis.to_crs(edges.crs)
gpath_p = gpath.to_crs(edges.crs)

# If you already filtered within city polygon in UTM earlier, re-do it␣
 ↪consistently here:
# Project city polygon to the SAME CRS as the road graph
city_m = city_gdf.to_crs(edges.crs)
city_poly_m = city_m.geometry.iloc[0]

# Now filter consistently
gvis_p  = gvis.to_crs(edges.crs)
gpath_p = gpath.to_crs(edges.crs)

gvis_p  = gvis_p[gvis_p.within(city_poly_m)]
gpath_p = gpath_p[gpath_p.within(city_poly_m)]
city_poly_m = city_m.geometry.iloc[0]

gvis_p  = gvis_p[gvis_p.within(city_poly_m)]
gpath_p = gpath_p[gpath_p.within(city_poly_m)]
```

[294]:
```
gpath_p = gpath_p.sort_values(["episode_id", "time"])
gpath_p["t_next"] = gpath_p.groupby("episode_id")["time"].shift(-1)

gpath_p["dt_min"] = (gpath_p["t_next"] - gpath_p["time"]).dt.total_seconds()/60.
 ↪0
gpath_p["dt_min"] = gpath_p["dt_min"].fillna(0)

# Clip instead of dropping: keeps info without letting long gaps dominate
```

```
gpath_p["weight"] = gpath_p["dt_min"].clip(lower=0, upper=30)

# remove zeros
gpath_p = gpath_p[gpath_p["weight"] > 0]
```

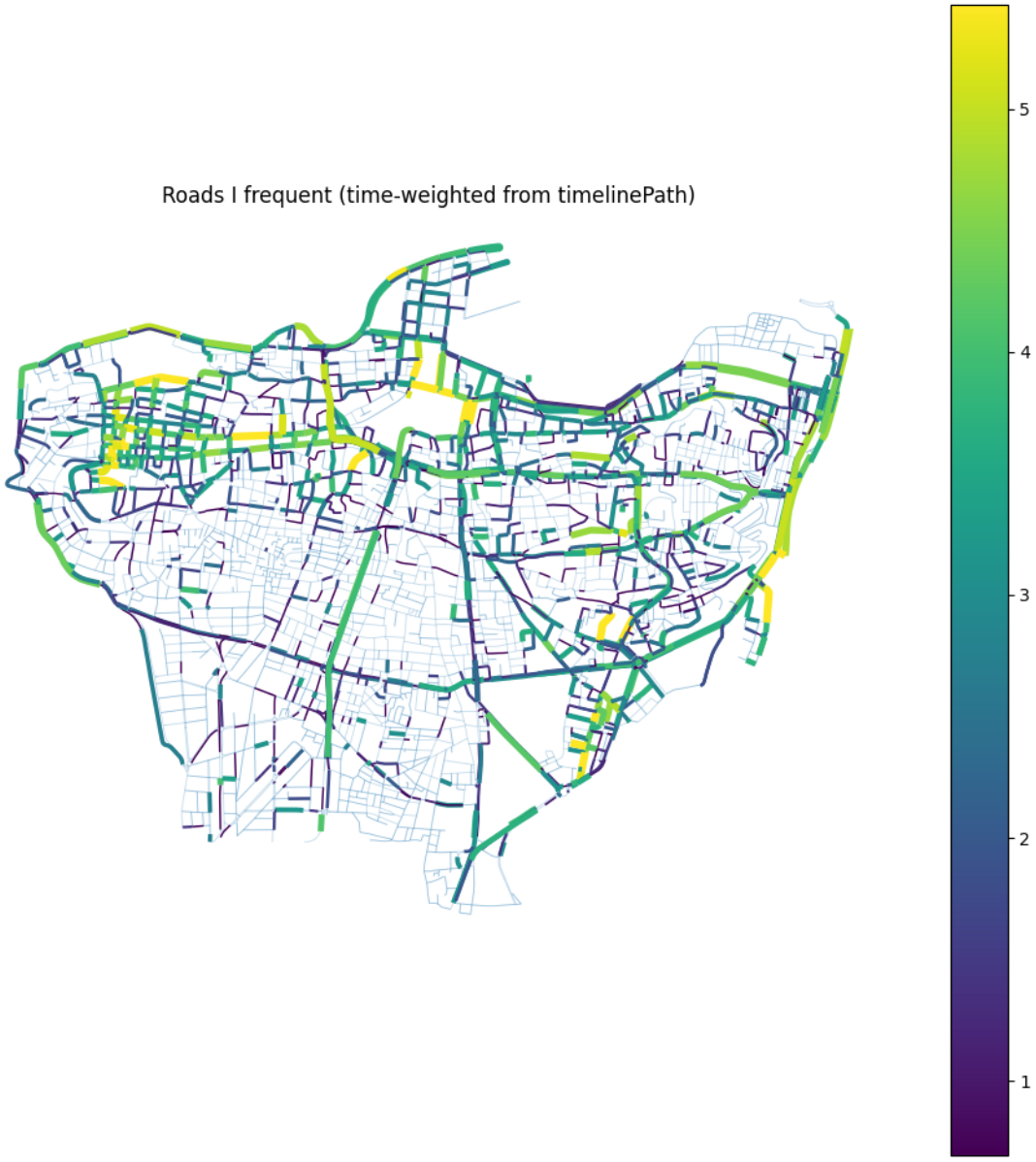
[298]:
```
uvk = ox.distance.nearest_edges(
    G,
    X=gpath_p.geometry.x.values,
    Y=gpath_p.geometry.y.values
)

# uvk sometimes comes back as (n,) list of tuples OR (n,1) array of tuples.
uvk_arr = np.asarray(uvk)

# Flatten if it's (n,1)
if uvk_arr.ndim == 2 and uvk_arr.shape[1] == 1:
    uvk_arr = uvk_arr[:, 0]

# Now uvk_arr should be length n, each element tuple-like
def unpack_edge(e):
    # e may be (u,v,key) or (u,v) depending on graph / osmnx version
    if isinstance(e, tuple):
        if len(e) == 3:
            return e
        if len(e) == 2:
            u, v = e
            return (u, v, 0)
    # fallback: if something weird happens
    return (np.nan, np.nan, np.nan)

uvk_unpacked = np.array([unpack_edge(e) for e in uvk_arr], dtype=object)

gpath_p["u"] = uvk_unpacked[:, 0]
gpath_p["v"] = uvk_unpacked[:, 1]
gpath_p["key"] = uvk_unpacked[:, 2]
```

[299]:
```
edge_move = (
    gpath_p.groupby(["u", "v", "key"])["weight"]
    .sum()
    .rename("w_move_min")
    .reset_index()
)
```

[302]:
```
# Join weights onto the edge GeoDataFrame
edges_w = edges.merge(edge_move, on=["u", "v", "key"], how="left")
edges_w["w_move_min"] = edges_w["w_move_min"].fillna(0.0)
```

```
# ensure numeric and non-negative (robust)
edges_w["w_move_min"] = (
    pd.to_numeric(edges_w["w_move_min"], errors="coerce")
    .fillna(0.0)
    .clip(lower=0.0)
)

# log scaling to tame outliers
edges_w["w_plot"] = np.log1p(edges_w["w_move_min"])

# clip extreme tail ONLY if there is variation
if edges_w["w_plot"].gt(0).any():
    q99 = edges_w.loc[edges_w["w_plot"] > 0, "w_plot"].quantile(0.99)
    edges_w["w_plot"] = edges_w["w_plot"].clip(upper=q99)
```

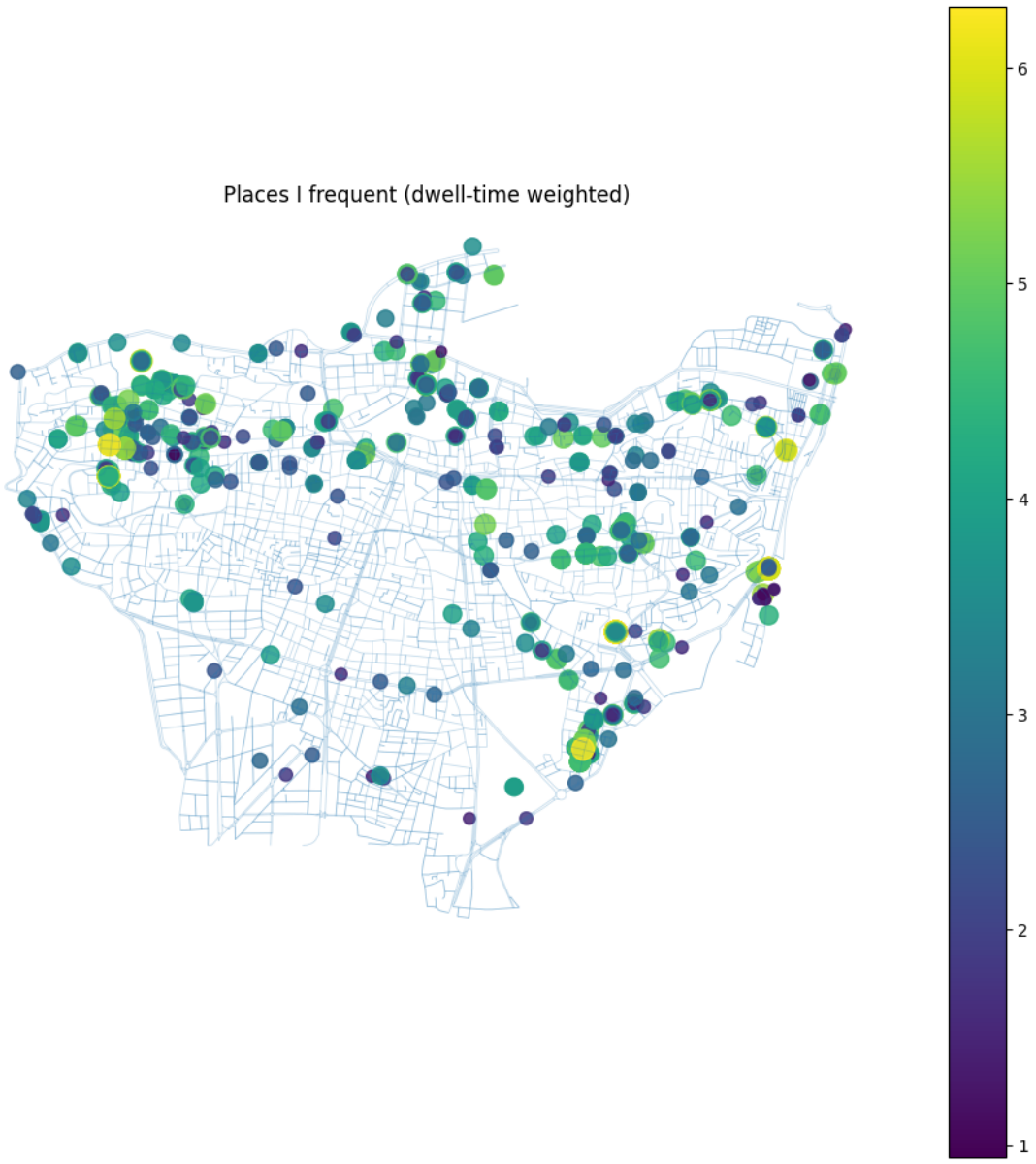
[303]:
```
fig, ax = plt.subplots(figsize=(12, 12))
ax.set_axis_off()

# Base roads (light)
edges.plot(ax=ax, linewidth=0.6, alpha=0.25)

# Weighted overlay: only plot edges with some weight
hot = edges_w[edges_w["w_move_min"] > 0]

# linewidth scaling
lw = 0.5 + 4.0 * (hot["w_plot"] / hot["w_plot"].max())

hot.plot(
    ax=ax,
    linewidth=lw,
    column="w_plot",
    legend=True,
)

ax.set_title("Roads I frequent (time-weighted from timelinePath)")
plt.show()
```



[304]:
```
gvis_p = gvis.to_crs(edges.crs)
gvis_p = gvis_p[gvis_p.within(city_poly_m)]

# If you merged place_id groups already, keep that. Otherwise group again:
places = gvis_p.copy()
places["w_place"] = places["weight"]  # dwell * prob
```

[305]:
```
places["w_plot"] = np.log1p(places["w_place"])
q99p = places["w_plot"].quantile(0.99)
places["w_plot"] = places["w_plot"].clip(upper=q99p)
```

```
fig, ax = plt.subplots(figsize=(12, 12))
ax.set_axis_off()

edges.plot(ax=ax, linewidth=0.6, alpha=0.25)

# point size scaling
s = 10 + 150 * (places["w_plot"] / places["w_plot"].max())

places.plot(
    ax=ax,
    markersize=s,
    column="w_plot",
    legend=True,
    alpha=0.85
)

ax.set_title("Places I frequent (dwell-time weighted)")
plt.show()
```



[308]:
```
import osmnx as ox

pois = ox.features_from_polygon(city_poly, tags={"amenity": True, "shop": True,␣
 ↪"tourism": True})
pois = pois.to_crs(edges.crs)  # match graph CRS (robust)


# Use nearest POI for each visit point
joined = gpd.sjoin_nearest(
    places[["w_place", "geometry"]],
```

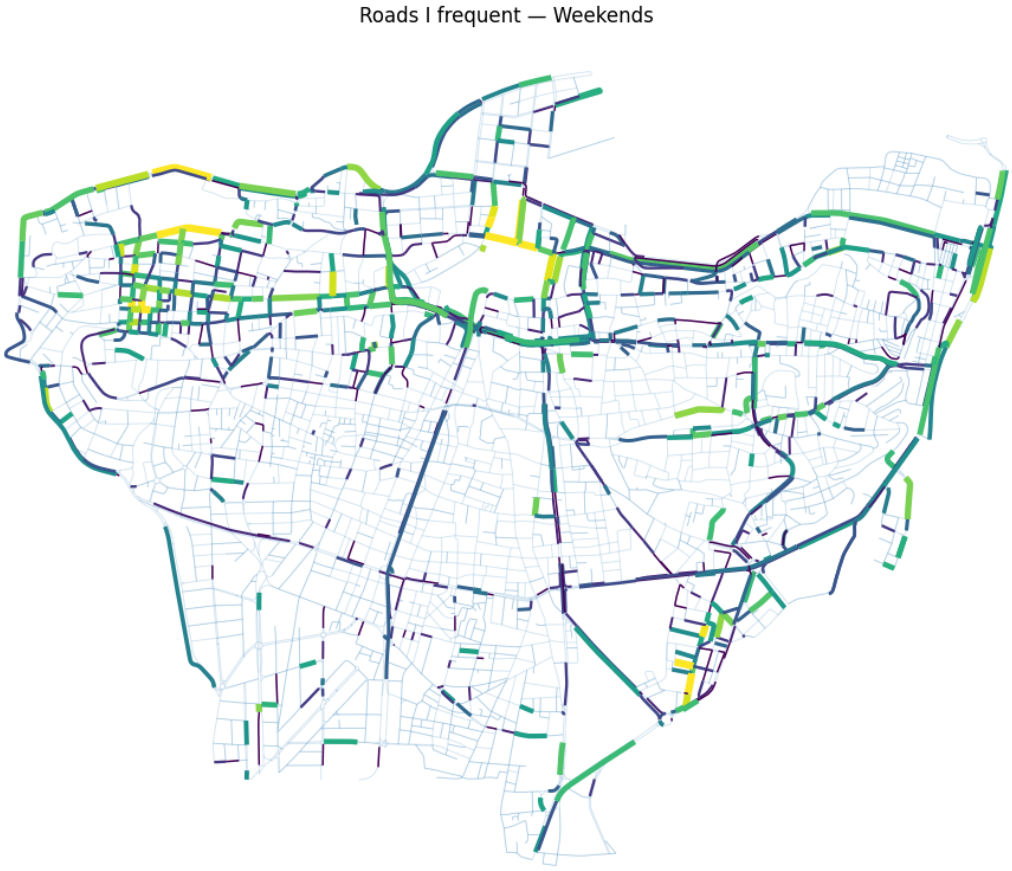
```
    pois[["name", "amenity", "shop", "tourism", "geometry"]],
    how="left",
    distance_col="dist_m"
)

poi_scores = (
    joined.groupby(["name","amenity","shop","tourism"])["w_place"]
    .sum()
    .sort_values(ascending=False)
)

poi_scores.head(20)
```

[308]:
```
Series([], Name: w_place, dtype: float64)
```

[309]:
```
gpath_p["weekday"] = gpath_p["time"].dt.weekday
gpath_p["is_weekend"] = gpath_p["weekday"] >= 5

def snap_edges_df(G, df_points):
    uvk = ox.distance.nearest_edges(G, X=df_points.geometry.x.values,␣
 ↪Y=df_points.geometry.y.values)
    uvk_arr = np.asarray(uvk)
    if uvk_arr.ndim == 2 and uvk_arr.shape[1] == 1:
        uvk_arr = uvk_arr[:, 0]

    def unpack_edge(e):
        if isinstance(e, tuple):
            if len(e) == 3:
                return e
            if len(e) == 2:
                u, v = e
                return (u, v, 0)
        return (np.nan, np.nan, np.nan)

    uvk_unpacked = np.array([unpack_edge(e) for e in uvk_arr], dtype=object)
    out = df_points.copy()
    out["u"] = uvk_unpacked[:, 0]
    out["v"] = uvk_unpacked[:, 1]
    out["key"] = uvk_unpacked[:, 2]
    out = out.dropna(subset=["u", "v", "key"])
    return out

for label, sub0 in [
    ("Weekdays", gpath_p[~gpath_p["is_weekend"]]),
    ("Weekends", gpath_p[gpath_p["is_weekend"]]),
]:
    sub = snap_edges_df(G, sub0)
```

```
    edge_move = (
        sub.groupby(["u", "v", "key"])["weight"]
        .sum()
        .rename("w")
        .reset_index()
    )

    edges_tmp = edges.merge(edge_move, on=["u", "v", "key"], how="left")
    edges_tmp["w"] = pd.to_numeric(edges_tmp["w"], errors="coerce").fillna(0.0).
↪clip(lower=0.0)

    edges_tmp["w_plot"] = np.log1p(edges_tmp["w"])
    if edges_tmp["w_plot"].gt(0).any():
        q99 = edges_tmp.loc[edges_tmp["w_plot"] > 0, "w_plot"].quantile(0.99)
        edges_tmp["w_plot"] = edges_tmp["w_plot"].clip(upper=q99)

    fig, ax = plt.subplots(figsize=(12, 12))
    ax.set_axis_off()
    edges.plot(ax=ax, linewidth=0.6, alpha=0.2)

    hot = edges_tmp[edges_tmp["w"] > 0]
    if len(hot) > 0:
        lw = 0.5 + 4.0 * (hot["w_plot"] / hot["w_plot"].max())
        hot.plot(ax=ax, linewidth=lw, column="w_plot", legend=False)

    ax.set_title(f"Roads I frequent - {label}")
    plt.show()
```


Roads I frequent — Weekdays


Roads I frequent — Weekends